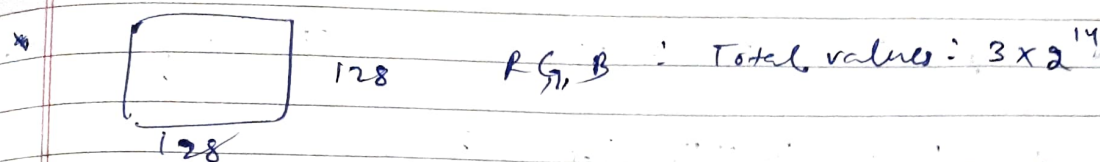


IMAGE CLASSIFICATION



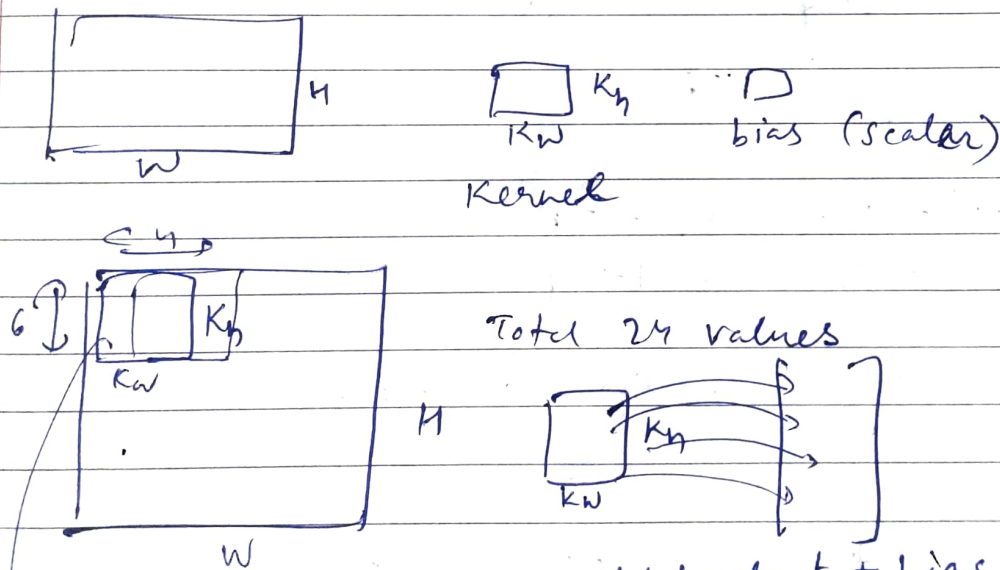
* Five way classification
 $w : 5 \times (3 \times 2^{14})$

Prediction vector : $w \times \bar{x} : 5 \times 1$

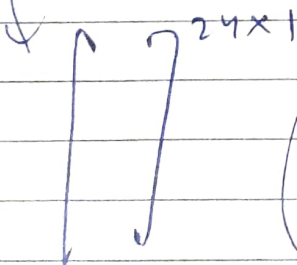
Softmax over the prediction vectors.

Limitations :- 1) 2D changed into 1D vectors (info lost),
 2) Linear transformation.

* Convolution operator over 2D data :-



dot product + bias



Example :- $W = 128$ $H = 64$ $K_w = 9$
 $K_h = 6$

Input :- $C_{in} \times H \times W$ $C_{in} = \text{no. of channels}$

filter : $C_{in} \times K_h \times K_w + \text{bias}$

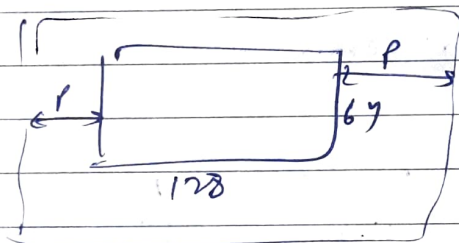
If multiple filters : $\text{Cont} \times C_{in} \times K_h \times K_w + \text{Cont bias}$

Output : $\text{Cont} \times H' \times W'$

$$H' = (H - K_h) + 1 \quad W' = (W - K_w) + 1$$

Why we need bias :- For getting zero output or non zero outputs

Padding helps to maintain size.



$$H' = H - K_h + 1 + 2P$$

$$W' = W - K_w + 1 + 2P$$

Generally : $K_h = K_w$ (square kernel)

To make $H = H' \Rightarrow K_h = K_w = 2P + 1$

Larger stride scales down the representation -

$$H' = \left(\frac{H - K_h + 2P}{s} \right) + 1$$

$$W' = \left(\frac{W - K_w + 2P}{s} \right) + 1$$

For 2D Convolution:

$C_{in} \times L$: input

$Cont \times C_{in} \times K$: Conv. layer.

- * Stride reduces the resolution. (one point is loss of info)
- * Padding increase resolution.
- * Padding introduces noise.
- * K_h and K_w : Create summary of tile (again loss of info)

$$C_{in} \times H \times W \xrightarrow{+P} \left[\begin{array}{c} Cont \times C_{in} \times K_h \times K_w \\ + \\ S, \text{ bias vector of } Cont \end{array} \right] \rightarrow Cont \times H' \times W'$$

$Cont$ = no. of layers/channels we want in output.

* A sequence of convolutions is still linear.

* Introduce non-linearity
 ↳ Activation
 ↳ Pooling

* Avg. Pooling

* Min, Max ~~Conv~~ Pooling

Pooling window of 2×2 reduces size to $1/4$.

Each channel is still preserved.

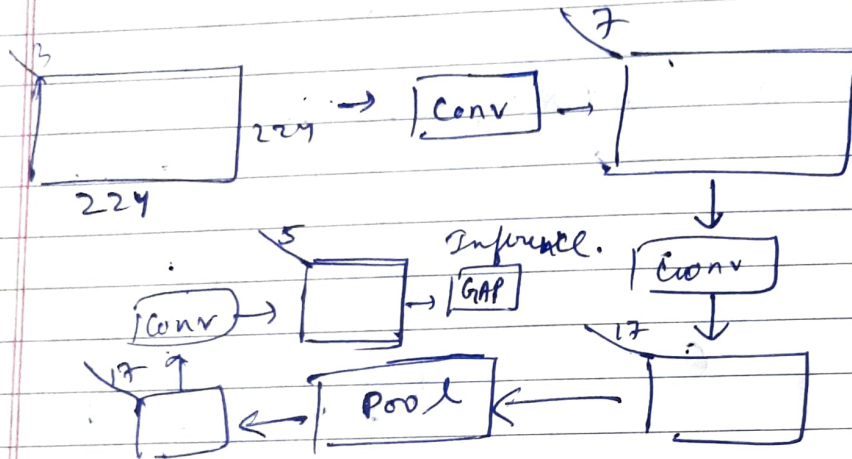
* GAP $\rightarrow$ (Global avg Pooling)

GAP $\Rightarrow$ changes $\Rightarrow H \times W \times C \Rightarrow 1 \times 1 \times C$
(averages the whole map)

* Inference is performed on final vector:-

- GAP
- Flattening
- Fully connected layer or more complex classifier.

* ALEXNET :-

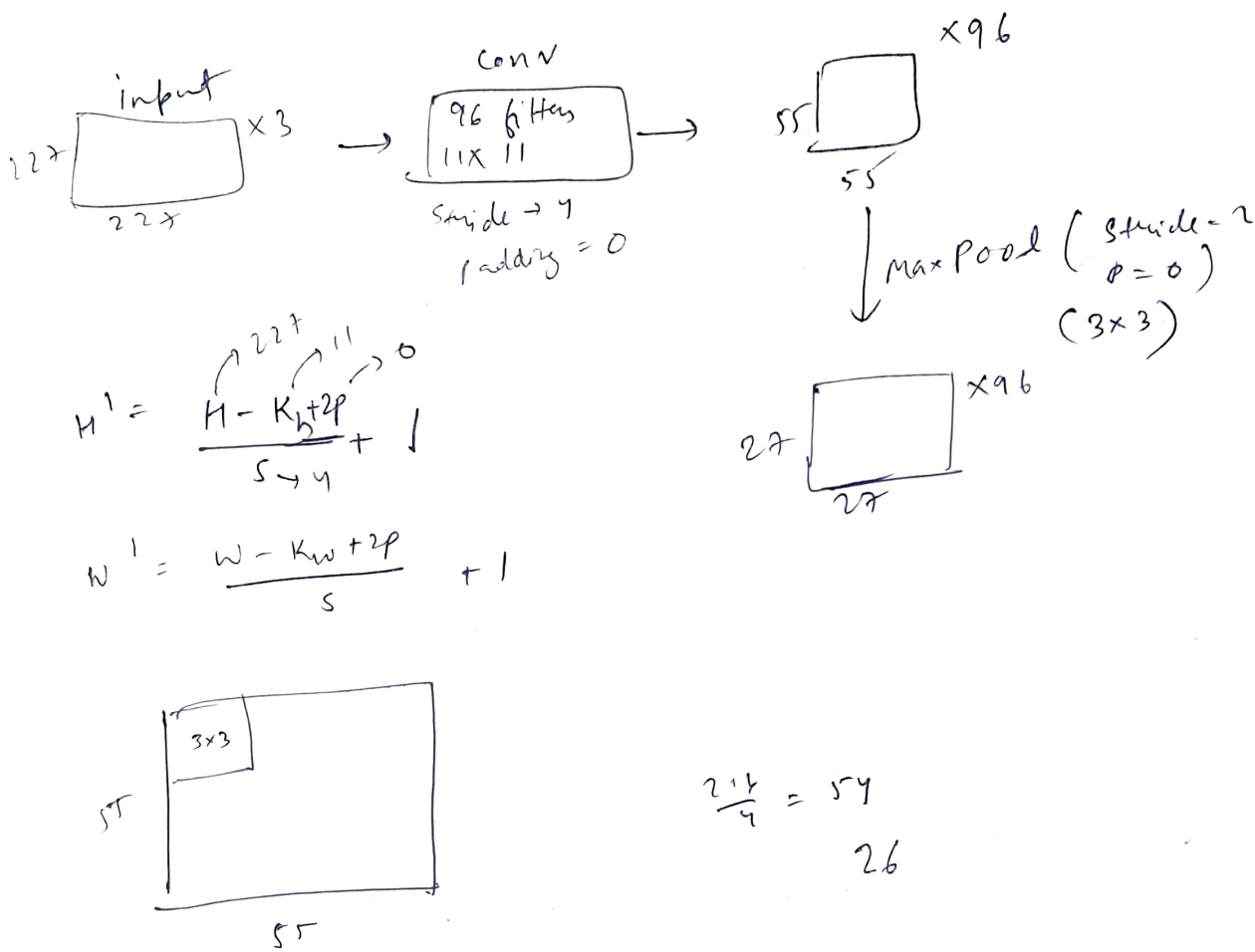


ALEXNET

* Objects in realistic settings exhibit considerable variability so to learn to recognise them it is necessary to use much larger training sets.

* To learn thousands of objects from millions of images, we need a model with a large learning capacity.

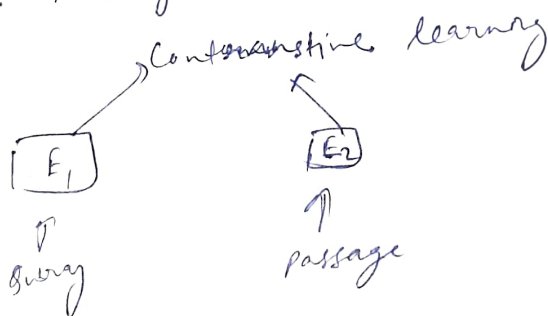
Alexnet Architecture



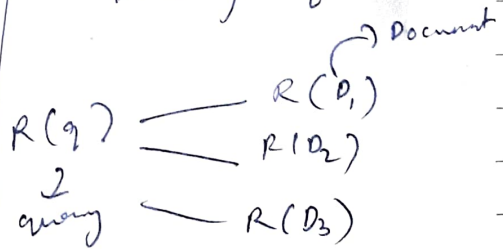
Problems with DL models

1. Static knowledge (Rag helps here)
2. Difficult updates $\rightarrow$ Niche topics
3. Hallucinations.

Dense Passage Retriever



During inference:-



calculate similarities during inference.
 vector databases are used for this computing.

- * BART → Denoising Transformer
 - Can be easily finetuned for any task.

* RAG Architecture:-

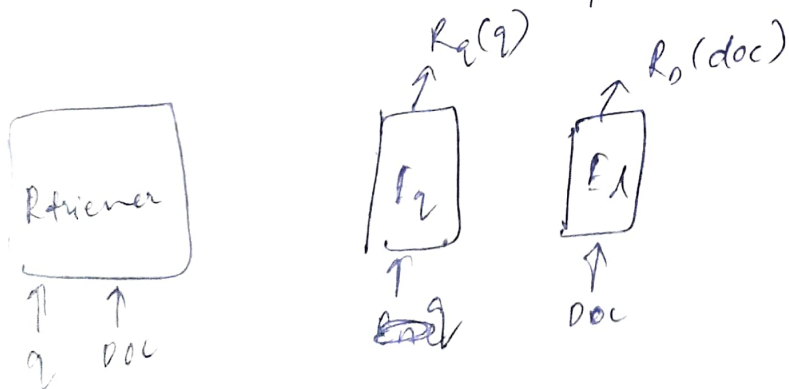
- Retriever → Find top K docs
- Generator → Augment top K docs with query and give it to BART

RAG ≠ Prompt Concatenation:-

- Prompt with one doc at a time.
- Conceptually run K such sessions.
- Generate token probabilities
- Combine

$$p(y/n) = \sum_{z \in \text{top } K} p(z/n) \cdot p_{\theta}(y/n, z)$$

$$\prod_i p_{\theta}(y_i/n, z, y_{1:i-1})$$



RAG-seq
↓
RAG-token

OPR

OPR → only worked for QA tasks

↓
FiD
↓
ATLAS

OPR

Contriever → works good for any NLP task.

↓
LCM based RAG

Contriever - unsupervised learning.

q_1 $a_{1,1}$ $a_{1,2}$... $a_{1,n}$

$[q_1 + d_1] \rightarrow a_{1d1,1} \quad a_{1d1,2} \quad \dots \quad a_{1d1,n}$

$[q_1 + d_2] \rightarrow a_{1d2,1} \quad a_{1d2,2} \quad \dots \quad a_{1d2,n}$

In RAG seq! → $[q_1 + d_1] \rightarrow \dots \rightarrow \sum$
 $[q_1 + d_2] \rightarrow \dots \rightarrow \sum$
 Add these 2 together
 these q are independent of each other

In RAG token :- $[q_1 + d_1]$
 $[q_1 + d_2]$
 $a_{1d123} \rightarrow q_1 + d_1$
 $q_2 + d_2$
 $q_1 + d_3$

* In RAG token → fusion happens across docs.

* FID → Forget about $P(z/n)$ (2021) this is a param learnt by itself.

- large cross attention matrix
- Model learns $P(z/n)$ through attention.
- small encoder, Large decoder

↓
quadratic
cross attention.

↓
linear cross
attention

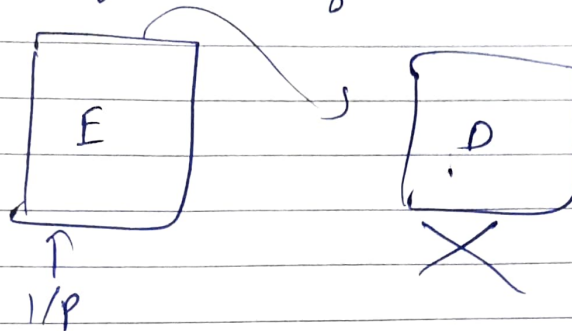
- More documents, better performance.

* ATLAS: Multitask RAG (2022) →

- Retriever: retriever
 - ↳ Unsupervised training.
- Training over hundreds of tasks.
- Small model RAG $\gg$ Large model for a variety of tasks.
- Generator: Few shot learner $\gg$ (doubt)

→ CLS token

→ BERT :- used for classification.



masked Language Model → 15%

↳ 1.5% wrong

↳ 1.5% unchanged

→ 12% masked.

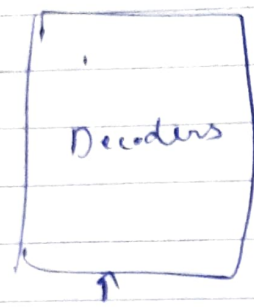
→ we won't take the input embedding instead will take something else.

Double → How can I use Bert to do generative task.

Advantages :- less computational task.

Disadvantages → less variety of tasks can be done.

* GPT - Decoder



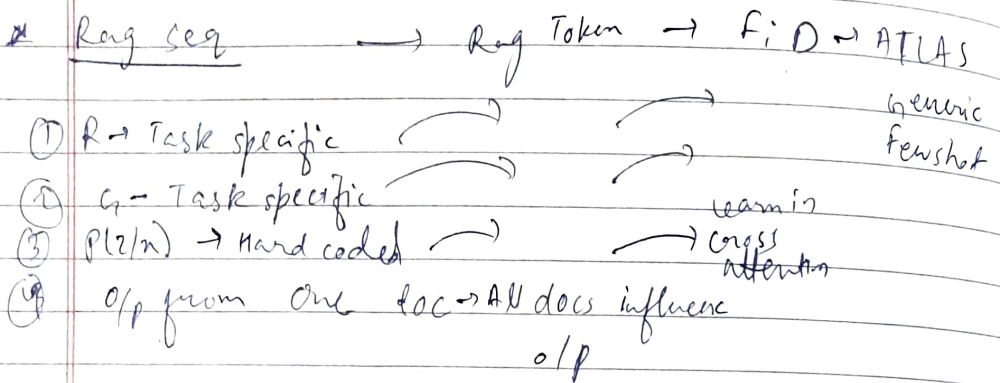
directly text input, not embedding.

Norm + RC
FFNN

X Norm + RC
X Cross Attention
Norm + Res Conn
MSA

Disadvantages → Cross Attention block is gone.
But tradeoff is good.

* More ^{pre-} training = better downstream performance, even though it's expensive.



LLM based Rag
zero shot.

* OPTIMIZATION

* Momentum : Polyak (look at history first)
Nesterov (look at current first)

* Ada Grad :-

* RMS Prop :-

(Jeffrey Hinton :)

* Adam :-

* Adam W :-

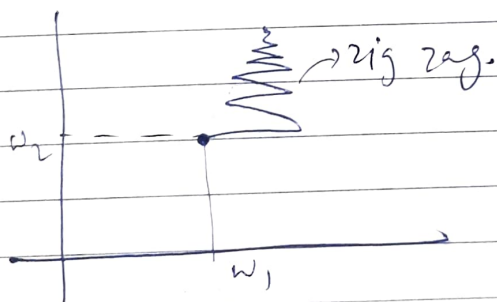
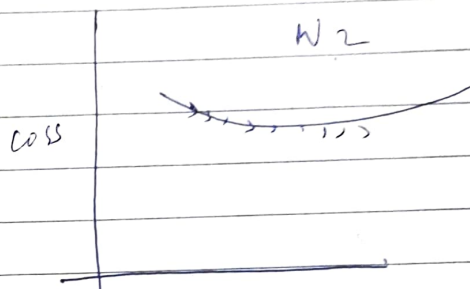
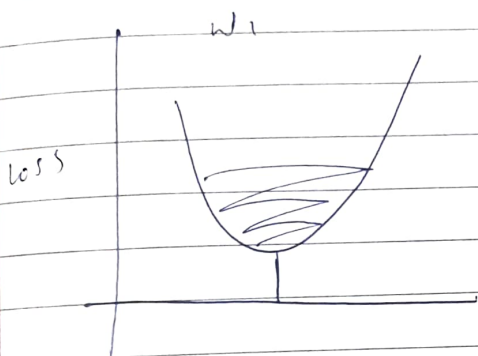
* Gradient Descent : No history

$$\theta_{t+1} = \theta_t - \nabla_{\theta} L(\theta_t)$$

steep direction : oscillation

shallow dir : No movement

zig zag path to the minima.



* SGD with momentum : History (Polyak)

$$V_t = \beta V_{t-1} + (1-\beta) \nabla_{\theta} L(\theta_t)$$

$$0 < \beta < 1$$

$$\theta_{t+1} = \theta_t - \eta V_t$$

→ Exponentially weighted moving average of past gradients.

→ Smoother convergence, empirically.

→ No guarantee of being better than SGD plain.

Polyak : Gradient $\rightarrow$ Momentum $\rightarrow$ update.
Nesterov : look ahead gradient $\rightarrow$ momentum

classmate

Date

update

* Nesterov :-

→ doubt

$$V_t = \beta V_{t-1} + (1-\beta) \nabla_{\theta} L(\theta_t - \eta \beta V_{t-1})$$

$$\theta_{t+1} = \theta_t - \eta V_t$$

* Adagrad :-

$$\textcircled{1} \theta_{t+1,i} = \theta_{t,i} - \eta_{t,i} \cdot g_{t,i}$$

$$\textcircled{2} g_{t,i} = \frac{\partial L(\theta_t)}{\partial \theta_i}$$

$$\textcircled{3} \eta_{t,i} = \frac{\eta}{\sqrt{S_{t,i}} + \epsilon} \quad (\text{with time } \eta_{t,i} \text{ tends to 0.})$$

$$\textcircled{4} S_{t,i} = S_{t-1,i} + g_{t,i}^2 \quad (\text{always go up.})$$

* RMS Prop : Maintain EWMA of second moment.

$$\rightarrow S_{t,i} = \beta (S_{t-1,i}) + (1-\beta) g_{t,i}^2 \rightarrow \text{recent gradients matter more than older gradients.}$$

$$\rightarrow \eta_{t,i} = \frac{\eta}{\sqrt{S_{t,i}} + \epsilon}$$

$$\rightarrow \theta_{t+1,i} = \theta_{t,i} - \eta_{t,i} \cdot g_{t,i}$$

→ $S_{t,i}$, does not increase monotonically

Adam $\rightarrow$ better generalisation

$$\rightarrow \theta_{t+1,i} = \theta_{t,i} - \eta_{t,i} \hat{m}_{t,i} - \alpha \Delta \theta_{t,i}$$

classmate

Date

Page

60/2

0.8/2

Adam : Adaptive step size + Directional smoothing

$$g_{t,i} = \frac{\partial L(\theta_t)}{\partial \theta_{t,i}}$$

$$m_{t,i} = \beta_1 m_{t-1,i} + (1-\beta_1) g_{t,i}$$

$$\theta_{t+1,i} = \theta_{t,i} - \eta_{t,i} m_{t,i}$$

$$v_{t,i} = \beta_2 v_{t-1,i} + (1-\beta_2) g_{t,i}^2$$

$$\beta_2 > \beta_1$$

$$\beta_1 = 0.9$$

$$\beta_2 = 0.999$$

$$\eta_{t,i} = \frac{\eta}{\sqrt{v_{t,i}} + \epsilon}$$

Numerically boosting $\hat{m}_{t,i}$ and $\hat{v}_{t,i}$

Can't use regularisation directly with Adam.

Memory footprint of optimizer :-

No. of params $4 \times$

Gradient $4 \times$

First moment (m_t) $4 \times$

2nd moment v_t : $4 \times$

we can reduce this by ignoring 1st moment and approximating 2nd moment.

Total 16X bytes memory.

Adafactor :- $O(mn)$ to $O(m+n)$

We only need to maintain relative step size across the params.

$$\sqrt{[m][n]} = \frac{\sigma_m(n)}{\text{mean}(g^2)}$$

$\rightarrow (m+n)$ size instead of $m \cdot n$.

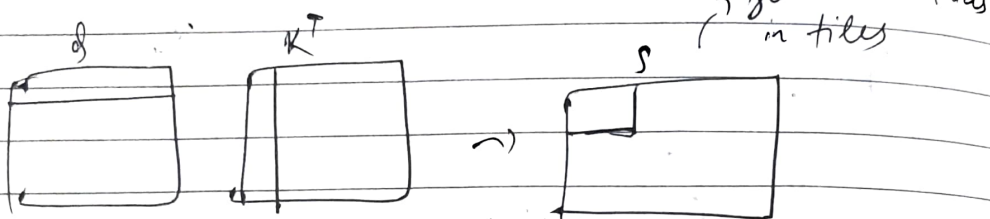
$\hookrightarrow$ same on memory

$$\hat{m}_{t,i} = \frac{m_t}{1-\beta_1}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2}$$

* GPUs have similar problem.

* Flash Attention :-

→ 1st version :-



(Take 1 row of Q and subset of K^T whatever space is left in SRAM of GPU.

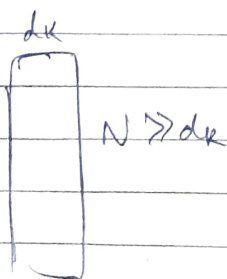
(Directly do: $(QK^T)V$.
 (don't wait for computing whole matrix,

*

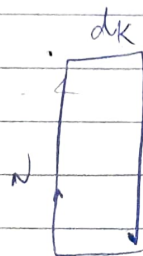
$$\begin{array}{ccc}
 Q & K & V \\
 & \swarrow \quad \searrow & \\
 & 512 \quad 64 & \\
 & N \times d_K & \\
 & \downarrow & \\
 & 10^6 \times 6 = 10^{12} & \\
 & \downarrow & \\
 & N^2 & \\
 & \downarrow & \\
 & 10^6 \times 6 = 10^{12} &
 \end{array}$$

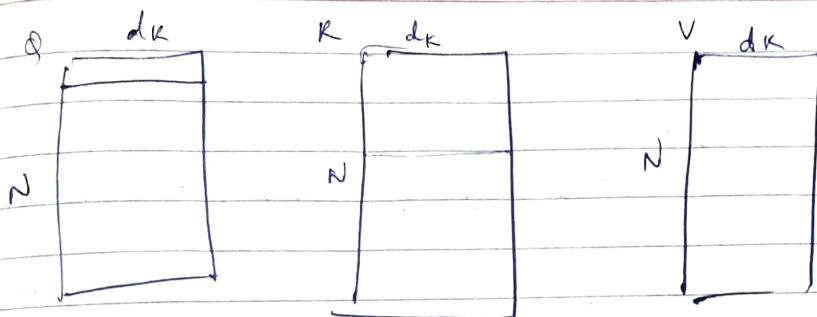
Q matrix.

K matrix N



$V \rightarrow$





$S \rightarrow (QK^T)_{N \times N} \Rightarrow$ can't store

Take a row from S and col of V .

* Naive softmax :- e^x and sum might have overflow problem.

* safe softmax $\rightarrow \frac{e^{x(i) - \max}}{\sum_{i=1}^n e^{x(i) - \max}}$ (in 3 passes)

* safe softmax in 2 passes :-

$$\text{sum}_i, \text{max}_i = f(\text{sum}_{i-1}, \text{max}_{i-1})$$

$$\text{sum}_i = \left(\sum_{j=1}^{i-1} e^{x(j) - \text{max}_{i-1}} \right) e^{\text{max}_{i-1} - \text{max}_i} + \frac{e^{x(i) - \text{max}_i}}{\sum e^{x(j)}}$$

Using this we don't need a whole row of S to do softmax.

NOTE $\rightarrow$ LOMO (Low Memory optimization)
read about it.

* Regularization :-

$$L_r = L(\theta) + \text{Regularization loss}$$

$$L_2 \text{ regularization : } L_{\text{total}} = L_{\text{data}} + \frac{\lambda}{2} \| \theta \|^2$$

Squared L2 norm

$$\nabla L_{\text{total}} = \nabla L_{\text{data}} + \lambda \theta$$

$$\begin{aligned} \text{SGD: } \theta_{t+1} &= \theta_t - \eta (\nabla L_{\text{data}_t} + \lambda \theta_t) \\ &= (1 - \eta \lambda) \theta_t - \eta \nabla L_{\text{data}_t} \end{aligned}$$

* Large weights $\rightarrow$ Model sensitive to small changes in input.

* Small weights $\rightarrow$ Smoother decision boundaries

* Overfitting : Works on training data.
Fails on test data.

Vector Databases :- to store simple data. (Key-Value pairs)
 → for complex data $\Rightarrow$ RDBMS
 →

↓
vector

→ complexity of Databases $\rightarrow$

RDBMS $\rightarrow$ Key-Value $\rightarrow$ Vectors.

→ frequent operations :-

→ Equality range in RDBMS

→ search Key in key-value pairs.

→ vector: ~~vector~~ operations find ANN.

→ Is this key present or not $\rightarrow$ to answer this use hash table.

		<u>SISD</u> <u>CPU</u>	<u>SIMD</u> <u>GPU</u>
N vectors		d	1
d dim	$\uparrow$		
	$q(N)$	Nd	N

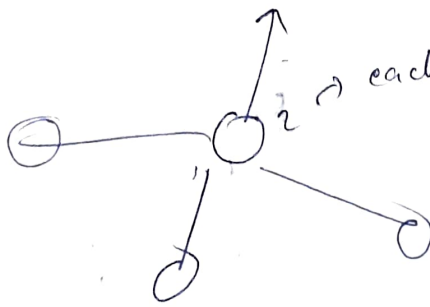
★ If we start with a heap of size K ~~at~~, how will you proceed??

→ make clusters of ~~size~~ $(m)^n$ of size $\frac{N}{m}$.

$$TC \rightarrow md + \left(\frac{N}{m}\right)d \cdot m'$$

m' = no. of shortlisted clusters.

$$\Rightarrow M \ll N$$



→ each datapoint connected to
Top. 50 + 5 neg samples

Beam search

✓

No SQLRelationalKey valueVector

Tree

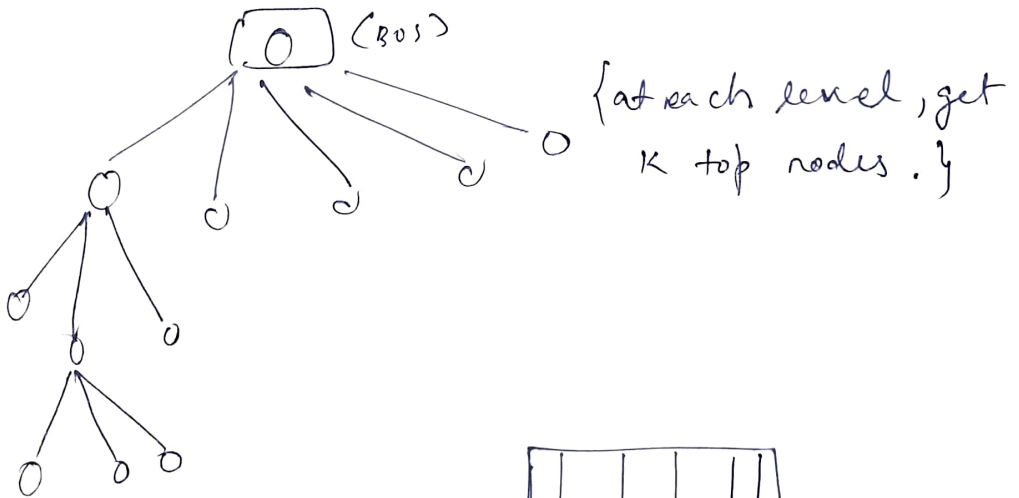
Hash

top K

Equality = $O(\log N)$ Equality = $O(1)$ Range = $O(\log N)$

- * Dense vs Sparse index
- * ordered vs unordered index.

* Beam Search :-



Candidates (top K)
(max heap)

Best so far
(min heap)

- * no. of edges is reduced $\Rightarrow$ then info is lost.
- * we can reduce the no. of vertices so that info is NOT lost.